Learning to Solve Capacitated Arc Routing Problems by Policy Gradient

Han Li [1] and Guiying Li [2,1]
[1]*School of Computer Science and Technology*,
University of Science and Technology of China, Hefei, China
lihan000@mail.ustc.edu.cn
[2]*Shenzhen Key Laboratory of Computational Intelligence*,
Department of Computer Science and Engineering,
Southern University of Science and Technology, Shenzhen, China
ligy@sustech.edu.cn

***Abstract*—Most heuristic algorithms for NP-hard combinatorial optimization problems require expertise in both the problem domains and heuristic methods. Recent research has begun to apply Deep Neural Network to learning heuristics for combinatorial optimization problems automatically. These works mainly focus problems with simple formulations, such as Travelling Salesman Problem and Vehicle Routing Problem defined on Euclidean graphs. This paper presents a novel deep reinforcement learning based algorithm for the Capacitated Arc Routing Problem which is defined on more complex non-Euclidean information graphs. The proposed approach is a combination of a Graph Convolutional Network and two encoder-decoder models. By regrading the negative objective values of CARP instances as the rewards, the proposed method optimizes the parameters with REINFORCE algorithm. In empirical experiments, the proposed method is able to generate solutions approximate optimal solutions well with much less time than heuristic algorithms.**

***Index Terms*—reinforcement learning, deep neural network, capacitated arc routing, optimization**

I. Introduction

Combinatorial optimization problems arise in extensive fundamental research and industry applications. In traditional methods, heuristic algorithms are most used to solve combinatorial optimization problems. However, designing a heuristic algorithm is extremely difficult because it requires specific knowledge in both problem domains and heuristic methods, which brings great challenges to designers of algorithms for various combinatorial optimization problems in real-world applications. Besides, heuristic algorithms solving problems by trial-and-error iterative procedures which is time-consuming.

Recently, deep learning based methods have achieved state-of-the-art performances in many applications, such as image classification [1], machine translation [2], and playing video games [3], etc. The success of deep learning provides another approach to solve combinatorial optimization problems. That is, by learning from the training data, the Deep Neural Network (DNN) can discover some heuristics for combinatorial optimization problems, thus requiring less well-designed artificial rules. Furthermore, deep learning based methods can speed up the solving process with some parallel hardware platforms such as GPUs and FPGAs which cannot be utilized by traditional algorithms. Actually, some research have already applied deep learning to solving combinatorial optimization problems, including Travelling Salesman Problem (TSP) [4]–[6] and Vehicle Routing Problem (VRP) [7]. To some extent, both TSP and VRP are too simple to be applied in practice directly.

This paper focuses on the Capacitated Arc Routing Problem (CARP), a more complex combinatorial optimization problem that has a lot of applications in reality, such as waste collection, mail delivery, and bus routing, etc [8]. CARP requires to find a set of routes with the minimal cost on a given information graph subject to certain constraints. Compared with TSP and VRP in above-mentioned works, CARP is more complex because:

1) TSP and VRP are defined on graphs represented with 2D Euclidean coordinates, while CARP is defined on a more complex non-Euclidean information graph. When DNN is applied, non-Euclidean information makes it very hard to construct numerical data representation for CARP instances.
2) CARP involves more constraints than TSP. Under those constraints, the solution space of CARP can be much more irregular than TSP. As proved in [9], a CARP instance with n task arcs can be transformed into a capacitated VRP instance with $3n + 1$ nodes, which reflects the greater inherent difficulty of solving CARP.

Considering the above two difficulties, this paper presents a novel neural network based solver optimized by reinforcement learning (NSRL) which is a combination of a modified Graph Convolutional Network (GCN) [10] and two encoder-decoder models. The procedure of solving CARP can be taken as a set-to-sequence prediction problem in which the input is a set of all task edges and the output is a sequence of those task edges. The architecture of NSRL combines a GCN and two encoder-decoder models in a serial manner. The GCN firstly reads the information graph of the CARP instance and outputs low-dimensional embeddings for all task edges of the instance. After that, the first encoder-decoder model with an pointer attention mechanism [11] reads the set of task edges embeddings and outputs a sequence as the initial solution of

978-1-7281-2153-6/19/$31.00 ©2019 IEEE

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

the instance. The second encoder-decoder model determines the directions of all edges in the initial solution. Supervised learning requires plenty of labeled data is not suitable for training the DNN-based solver because it is costly to obtain the labels, i.e., the optimal solutions of CARP instances. The proposed method uses reinforcement learning to optimize the parameters of the neural network model and takes the negative route length as the reward. To be specific, the entire model is parameterized as a route generating policy and is trained with the well-known REINFORCE algorithm [12].

The contributions of this paper are summarized in two aspects. First, NSRL is the first attempt to apply DNN model trained by reinforcement learning to solve CARP. Second, NSRL can generate a solution for an instance in much less time than heuristic algorithms by utilizing GPUs. The numerical experiments show NSRL can generate solutions that close to the optimal solutions.

The rest of the paper is organized as follows. The related works of neural optimization and some preliminaries of CARP are introduced in Section II. In Section III, the proposed method is described in details. Section IV analyzes the experimental studies. Section V concludes the paper.

II. Related Work and Background

A. Related Work

The research on applying neural networks for combinatorial optimization problems dates back to Hopfield Networks [13]. This work, along with several related research from the same period, such as Elastic Net [14] and Self-Organizing Feature Map [15], solves TSP by "learning from scratch" for each instance. These neural networks are totally different from DNNs that use data-driven learning procedure.

Pointer Network (Ptr-Net) [4] is the first DNN-based sequence-to-sequence model applied for combinatorial optimization problems. With the help of an attention mechanism, the decoder generates an element pointing to an input sequence index at each decoding step. That is, the output sequence is a permutation of the input sequence. The functionality of Ptr-Net is naturally suitable for some permutation-based combinatorial optimization problems, like TSP. The authors use Ptr-Net to solve 2D Euclidean TSP and optimize the parameters with supervised learning, that requires a large number of labeled training data. Actually, obtaining labeled data for combinatorial optimization problems means getting the optimal or near-optimal solutions for problem instances. However, it is nontrivial to solve combinatorial optimization problems, especially for some large-scale instances. Hence, supervised learning is replaced with reinforcement learning that uses actual objective values of combinatorial optimization problems as reward signals in the subsequent research. The work of [5] introduces the Actor-Critic algorithm [16] and regards the negative tour length as the reward. Their experiments show that training with reinforcement learning outperforms the supervised learning in [4] significantly on 2D Euclidean TSP instances.

The research of [7] focuses on VRP. This work replaces the encoder with an embedding layer to eliminate the effect of the order of the input sequence on the solution. Compared with TSP, VRP has more complex definitions and constraints. Thus, the method proposed in [7] divides each node of the VRP instance into two parts, static element and dynamic element, where static element is the immutable feature of the node and the dynamic element will change during the inference. The model is trained by Actor-Critic algorithm. The solutions generated by this method are inferior to the optimal solutions within 10% on VRP instances with 10 customers. On VRP instances with 20 customers, the gap is no more than 13%.

Structure2Vec Deep Q-learning (S2V-DQN) [6] uses a graph embedding model, named Structure2Vec (S2V) [17] as the neural network architecture for some graph-based combinatorial optimization problems. The S2V generates embedding for each node on the given graph. S2V-DQN proposes a parameterized evaluation function $Q(S, a)$ using the embeddings from S2V to evaluate the value that adds node a to the partial solution S. The algorithm adopts a greedy policy to generate the solution by adding the node with highest Q-value to current partial solution S at each step during the solution construction. For training, the authors use a Q-learning algorithm to learn the parameters of the policy. On simple graph-based combinatorial optimization problems, like Minimum Vertex Cover, Maximum Cut, and 2D Euclidean TSP, S2V-DQN can generate solutions very close to optimal solutions.

B. Notations and Problem Definition of CARP

This work considers the CARP defined on a graph $G(V, E, A)$, where V is the nodes set, E is the edges set, and A is the arcs set. For $\forall e \in E$, it has two directions which correcpond to $\{\overrightarrow{e}, \overleftarrow{e}\} \subset A$. $u_1(\overrightarrow{e})$, $u_2(\overrightarrow{e})$ denote the starting and the ending endpoints of $\overrightarrow{e}$, and $u_1(\overrightarrow{e}) = u_2(\overleftarrow{e})$, $u_2(\overrightarrow{e}) = u_1(\overleftarrow{e})$. There is a vehicle with the capacity Q. At the beginning, the initial location of the vehicle is the depot v_0, where $v_0 \in V$. For $\forall e \in E$, $dc(e)$, $d(e)$, and $sc(e)$ denotes the deadheading cost, the demand, and the serving cost, respectively. $dc(e) > 0$ and $0 \leq d(e) \leq Q$. If $d(e) > 0$, then e is a task edge and $sc(e) > 0$; otherwise, $sc(e) = 0$.

For the CARP instance on $G(V, E, A)$, a route is denoted as $R_i = \{a_1^i, a_2^i, ..., a_{|R_i|}^i\}$, where $a_j^i \in \{\overrightarrow{e_j^i}, \overleftarrow{e_j^i}\}$ denotes the direction that the vehicle serves the j-th task edge e_j^i in the route. The cost of the route R_i is formulated as:

$$\begin{aligned} C(R_i) = &\sum_{j=1}^{|R_i|} sc(a_j^i) + \sum_{j=2}^{|R_i|} dc\left(u_2\left(a_{j-1}^i\right), u_1\left(a_j^i\right)\right) \\ &+ dc\left(v_0, u_1\left(a_1^i\right)\right) + dc\left(u_2\left(a_{|R_i|}^i\right), v_0\right), \end{aligned} \tag{1}$$

where $dc(v_1, v_2)$ for $\forall v_1, v_2 \in V$ denotes the sum of deadheading cost of the shortest path generated by Dijkstra algorithm [18] between v_1 and v_2.

The solution consists of a set of routes, denoted by $y = \{R_1, R_2, ..., R_m\}$, where m is the number of routes. CARP

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

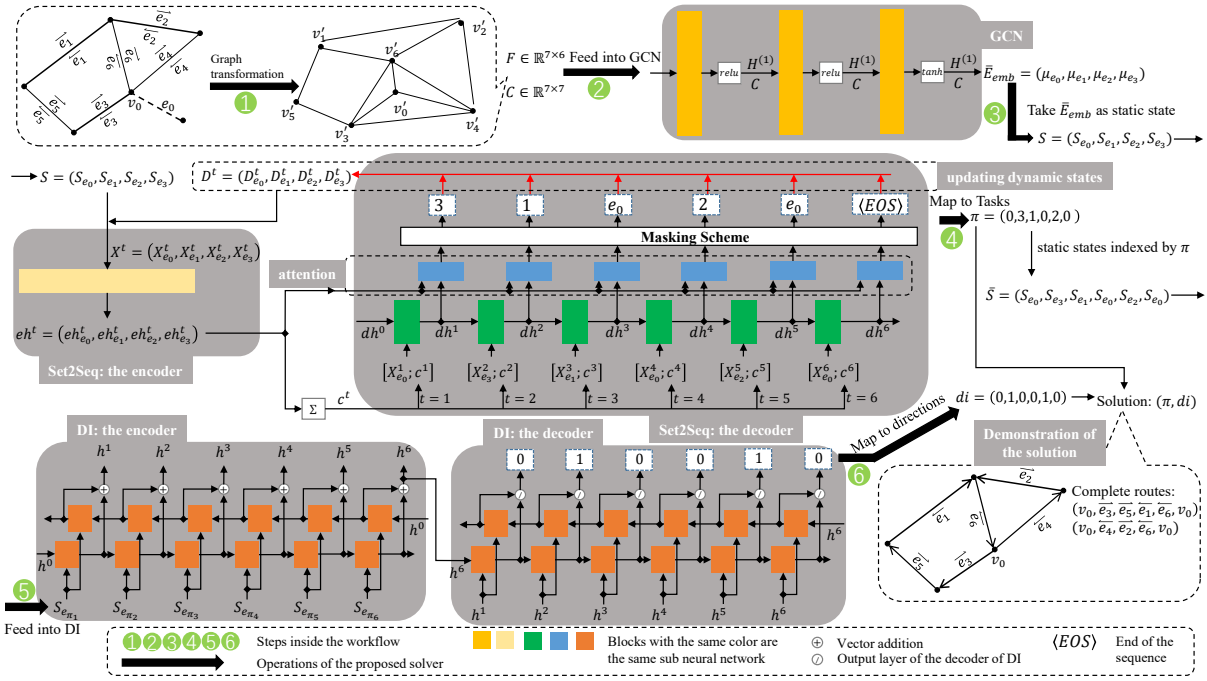


Fig. 1. Illustration of the whole solving process for a CAEP instance with NSRL. e_1, e_2, e_3 are three task egdes of the instance. The graph $G(V, E, A)$ of the instance is transformed into $G'(V', E')$ and the matrix F and the matrix C are generated (i.e., step 1 in the figure). The 3-layer GCN takes F and C as input to produce embeddings $\bar{E}_{emb}$ for task edges (i.e., step 2 in the figure). Then, $\bar{E}_{emb}$ that is considered as the static states S (i.e., step 3 in the figure) and the dynamic states D^t are sent into Set2Seq to output π_t at step t. The whole output of Set2Seq is a initial route sequence π (i.e., step 4 in the figure). At the last, DI generates directions di for task edges in π (i.e., step 5,6 in the figure). (π, di) is the final solution for the instance.

requires to find a optimal solution y^* with the minimal total cost $C(y^*) = \sum_{r \in y^*} C(r)$, subject to the following constraints:

1) The first endpoint and the last endpoint of each route must be the depot v_0;
2) The total demands of all edges in a route must not exceed the vehicle capacity Q;
3) Each task edge must be served once and only once, but it can be traversed without any limitations.

III. The Proposed Method

In this section, we first reformulate the solving of CARP as a set-to-sequence prediction problem and deploy a DNN parameterized routing policy to carry out this prediction. Then, the DNN model used in the proposed method is introduced.

For a CARP instance x, as described in II-B, the solution can be considered as a sequence of all routes connected one by one. The elements of the sequence are the depot and task edges. We can take the input as the set $x' = \{v_0, e_1, e_2, ..., e_T\}, d(e_i) > 0$, where T is the number of task edges, and the output as a sequence y which is a permutation of elements in x'. Thus, the solving of CARP is actually mapping the set x' into the sequence y, i.e., a set-to-sequence prediction problem.

Based on the set-to-sequence pattern, the solving process of NSRL can be divided into three main parts, graph representation with GCN, initial solution generation with Set2Seq model, and directions identification with Direction Indicator (DI). The deep models used in the three parts, i.e., GCN, Set2Seq, and DI, constitute the network architecture of the proposed method. GCN and Set2Seq define a parameterized policy to generate the initial solution. This policy is denoted as $p_S(\pi|x; \theta_S, \theta_G)$, where x is the CARP instance, π is the route in the initial solution, θ_G and θ_S are parameters of GCN and Set2Seq, respectively. DI defines a direction indicator policy $p_D(di|\pi, x; \theta_D)$, where di is directions of all task edges, and θ_D is the parameters of DI. Both of the two policies can be factorized by chain rule:

$$p_S(\pi|x; \theta_S, \theta_G) = \prod_{t=1}^{n} p(\pi_t|\pi_{1:t-1}, x; \theta_S, \theta_G), \tag{2}$$

$$p_D(di|\pi, x; \theta_D) = \prod_{t=1}^{n} p(di_t|\pi, x; \theta_D). \tag{3}$$

For the CARP instance x, the final solution given by NSRL is (π, di) which represents the route with exact edge directions. The entire model (the combination of GCN, Set2Seq, and DI) can be considered as a routing policy,

$$p(\pi, di|x; \theta_G, \theta_S, \theta_D) = p_S(\pi|x; \theta_G, \theta_S)\, p_D(di|\pi, x; \theta_D). \tag{4}$$

The illustration of a solving process for an instance with NSRL is shown in Fig 1. Details of NSRL are introduced in following sections.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

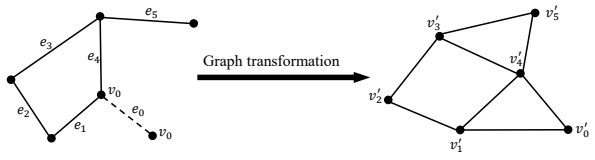


Fig. 2. Transformation for a graph: The left side is the original graph and the right side is the graph after transformation. The edges on the original graph become the nodes on the transformed graph. For a instance, e_3, e_4, and e_5 are three edges with the a same endpoint on the original graph. In the transformed graph, they are three interconnected nodes v'_3, v'_4, and v'_5.

A. GCN for Graph Representation

As described in last section, the input for the DNN model is a set of the depot and task edges. However, DNN can not take nodes or edges on non-Euclidean graphs as input directly. Therefore, NSRL adopts a graph embedding method, i.e., GCN [10] to convert the graph structure into numerical vectors as the input of the DNN. For $\forall v \in V$, GCN computes a l-dimensional embedding $\mu_v \in \mathbb{R}^l, l \ll |V|$. The information of the graph can be preserved with embeddings of nodes.

Actually, GCN aims at generating embeddings for the nodes on the non-Euclidean graph, which can not be used for edges, directly. However, the basic elements of CARP instances are edges, so that we make a transformation for the graph of the CARP instance before sending it to GCN. As illustrated in Fig 2, on the original graph, each edge contracts into a node and edges sharing a same node becomes neighborhood nodes. The depot v_0 on original graph can be regarded as a virutal task edge $e_0 = (v_0, v_0), d(e_0) = 0$ which is converted into node v'_0 in the transformed graph. The transformed graph is denoted as $G'(V', E')$ to distinguish it from the original graph. After the graph transforming, GCN can produce embeddings for nodes on the transformed graph (i.e., embeddings for edges on the original graph).

For the transformed graph $G'(V', E')$, let $m = |V'|$, $F \in \mathbb{R}^{m \times d}$ denote the feature description matrix of nodes, $C \in \mathbb{R}^{m \times m}$ denote graph structure matrix. GCN generates embedding matrix $E_{emb} \in \mathbb{R}^{m \times l}$ for nodes according to F and C. For the transformed graph of a CARP instance, the i-th row of F represents the feature of node v'_i (i.e., egde e_i on the original graph), formulated as:

$$F_i = \left(f(u_1), f(u_2), dc(u_1, v_0), dc(u_2, v_0), \frac{d(e_i)}{\sum_{e \in T} d(e)}, \frac{d(e_i)}{Q}\right) \in \mathbb{R}^{2l+4}, \tag{5}$$

where $u_1 = u_1(\overrightarrow{e_i}), u_2 = u_2(\overrightarrow{e_i})$ and $f(u) = 1$ if $u = v_0$, otherwise $f(u) = 0$.

The adjacency matrix is commonly used as C in GCN. However, solving a CARP instance strongly depends on distance between nodes on the graph which is not included in the adjacency matrix. For this sake, a customized matrix in which C_{ij} reflects the distance between v'_i and v'_j on G' (i.e., the distance between e_i and e_j on G) is used. We firstly construct an auxiliary matrix $\bar{C}$:

$$\bar{C}_{ij} = \begin{cases} \dfrac{1}{dhc(v'_i, v'_j)} & i \neq j, (v'_i, v'_j) \in E' \\ 0 & otherwise \end{cases}, \tag{6}$$

where $dhc(v'_i, v'_j) = dhc(e_i, e_j)$ is the deadheading cost between two edges defined in [19]:

$$dhc(e_i, e_j) = \frac{1}{4}\left[dc(u_1(\overrightarrow{e_i}), u_1(\overrightarrow{e_j})) + dc(u_1(\overrightarrow{e_i}), u_2(\overrightarrow{e_j})) + dc(u_2(\overrightarrow{e_i}), u_1(\overrightarrow{e_j})) + dc(u_2(\overrightarrow{e_i}), u_2(\overrightarrow{e_j}))\right], \tag{7}$$

where (u_1, v_1), (u_2, v_2) are the endpoints of e_i and e_j, respectively.

Then, each row in $\bar{C}$ needs to be normalized:

$$\bar{C}_i = \frac{1}{\sum_j \bar{C}_{ij}} \bar{C}_i. \tag{8}$$

Based on $\bar{C}$, $C = \bar{C} + 0.5I$, where I is the identity matrix.

NSRL uses a 3-layer GCN to generate embeddings and the computation of k-th layer is:

$$H^{(k)} = \sigma^{(k)}\left(CH^{(k-1)}W^{(k)}\right), \tag{9}$$

$H^{(k)}$ is the output of k-th layer, $W^{(k)}$ is the parameter matrix and $\sigma^{(k)}$ is the activation function. $H^{(1)} = F$, $\sigma^{(1)} = \sigma^{(2)} = relu$ and $\sigma^{(3)} = tanh$. The output of the last layer is the embedding matrix E_{emb} for edges. The subsequent models only need embeddings for task edges:

$$\bar{E}_{emb} = (\mu_{e_0}, \mu_{e_1}, ..., \mu_{e_T}), \tag{10}$$

e_0 is the virtual edge of the depot, $e_1, ..., e_T$ are task edges.

B. Set-to-Sequence model

There are two key characteristics in the solving process of CARP instances:

1) The solving is a set-to-sequence prediction problem.
2) The demands of task edges and the capacity of the vehicle are changing during the solution construction of a CARP instance. For example, if a task edge is served, then the capacity of the vehicle is subtracted by the its demand and its demand becomes 0.

Considering the first characteristic, Set2Seq based on an encoder-decoder model is applied to produce an initial solution for a CARP instance. The encoder is a linear transformation model rather than a Recurrent Neural Network (RNN) to avoid producing order information for the task edges set, and the decoder is a Gated Recurrent Unit (GRU) [20] network with a pointer attention mechanism [11] to produce the output sequence. This encoder-decoder model can make set-to-sequence predictions. Considering the second characteristic, Set2Seq splits the state of the task edge e_i into a static state S_{e_i} and a dynamic state D_{e_i} [7]. $S_{e_i} = \mu_{e_i} \in \mathbb{R}^l$ is constant throughout the solving process. $D_{e_i} = (d'(e_i), Q')$ where $d'(e_i)$, Q' are the real-time demand of e_i and the real-time capacity of the vehicle, will be updated at different decoding

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

steps. At the decoding step t, the dynamic state of e_i is $D^t_{e_i}$ and $X^t_{e_i} = \left[S_{e_i}; D^t_{e_i}\right]$ denotes the state of e_i.

To update and utilize the dynamic states of task edges in time, the encoding process alternates with the decoding process in the Set2Seq, which is different from encoder-decoder models in previous studies [20], [21]. To be specific, at step t, the encoder reads the states of task edges $X^t = \left(X^t_{e_0}, X^t_{e_1}, ..., X^t_{e_T}\right)$ and generates $eh^t = \left(eh^t_{e_0}, eh^t_{e_1}, ..., eh^t_{e_T}\right)$ where $eh^t_{e_i} \in \mathbb{R}^d$ is the encoder hidden state of e_i. The decoder firstly aggregates states of all task edges to get a context $c^t = \sum_{i=1}^{T} X^t_{e_i} \in \mathbb{R}^{l+2}$. Then the GRU inputs $\left[X^t_{e_{\pi_{t-1}}}; c^t\right] \in \mathbb{R}^{2l+4}$ where π_{t-1} the output of the decoder at step $t-1$ to generate a decoder hidden state $dh^t \in \mathbb{R}^d$. At the last, the pointer attention mechanism computes a probability distribution $prob^t$ over all task edges based on eh^t, dh^t and chooses the edge with the maximal probability as the output π_t at current step. The dynamic states of all edges are updated before the next step. The computational procedure of Set2Seq at step t is formulated as follow:

1) encoding:

$$eh^t = X^t W_e = \left(eh^t_{e_0}, eh^t_{e_1}, ..., eh^t_{e_T}\right), \tag{11}$$

where $W_e \in \mathbb{R}^{(l+2)\times d}$ is the parameter matrix of the encoder.

2) decoding:

$$\begin{aligned} c^t &= \sum_{i=1}^{T} X^t_{e_i}, \\ dt^t &= GRU\left(\left[X^t_{e_{\pi_{t-1}}}; c^t\right], dh^{t-1}\right), \end{aligned} \tag{12}$$

$$\begin{aligned} a^t_i &= tanh\left(eh^t_{e_i} W_{ref} + dh^t W_q\right) V, \\ a^t &= softmax(a^t), \\ ref &= a^t \cdot eh^t, \\ prob^t_i &= tanh\left(ref + dh^t W_q\right) V, \\ prob^t &= softmax\left(prob^t\right), \end{aligned} \tag{13}$$

$$\begin{aligned} \pi_t &= \arg\max_i prob^t_i, \\ p\left(\pi_t | \pi_{1:t-1}, x; \theta_S, \theta_G\right) &= prob^t_{\pi_t}. \end{aligned} \tag{14}$$

Where Equation (13) is the pointer attention mechanism, $W_{ref} \in \mathbb{R}^{d\times d}, W_q \in \mathbb{R}^{d\times d}, V \in \mathbb{R}^{d\times 1}$ are parameters of the pointer attention mechanism, $\cdot$ denotes dot product.

3) updating dynamic states:

$$\begin{aligned} Q' &= \begin{cases} Q' - d'(e_{\pi_t}) & \pi_t \neq 0 \\ Q & \pi_t = 0 \end{cases}, \\ d'(e_{\pi_t}) &= 0, \\ D^t_{e_i} &= \left(d'(e_i), Q'\right). \end{aligned} \tag{15}$$

Simply using the procedure above may result in generating infeasible solutions that violate the constraints of CARP. To fix that, we introduce a masking vector $mask$ for task edges to avoid the decoder choosing incorrect task edges at each step. At step t,

$$\begin{aligned} mask^t_0 &= \begin{cases} 0 & mask^{t-1}_0 = 1 \\ 1 & otherwise \end{cases} \\ mask^t_{i(i\in(1,...,T))} &= \begin{cases} 1 & 0 < d'(e_i) \leq Q' \\ 0 & otherwise \end{cases} \end{aligned}, \tag{16}$$

$mask^t_i = 0$ denotes that the task edge e_i should not be chosen at step t. After Equation (13), $prob^t_i$ is recalculated with

$$prob^t_i = mask^t_i \cdot prob^t_i. \tag{17}$$

The decoding step is finished if all masks are zero.

C. Direction Indicator

The output sequence π of Set2Seq is not a complete solution for CARP, for the elements in the output sequence are indices of all task edges. It just provides the positions of task edges in the route but lacks the directions ($\overrightarrow{e_i}$ or $\overleftarrow{e_i}$ for edge e_i) of task edges in which the vehicle passes through. Therefore, DI, a standard encoder-decoder [21] based model, is adopted to determine directions for task edges in π. Both the encoder and the decoder in DI are bidirectional GRU networks.

The encoder reads the static states $\left(S_{e_{\pi_1}}, S_{e_{\pi_2}}, ..., S_{e_{\pi_{|\pi|}}}\right)$ of task edges in the order given by π and outputs hidden states $h = \left(h_1, h_2, ..., h_{|\pi|}\right)$. The decoder reads elements of h one by one, produces a 2-dimensional probability vector for two directions, and choose the direction with the maximal probability as the output of each step. At step t, the computational procedure of the decoder of DI is as follows:

$$\begin{aligned} \bar{h}^t &= GRU\left(h_t, \bar{h}^{t-1}\right) \in \mathbb{R}^d, \\ prob^t &= sigmoid(\bar{h}^t W_o) \in (0-1)^2, \end{aligned} \tag{18}$$

$$\begin{aligned} di_t &= \arg\max_i prob^t_i, \\ p\left(di_t | \pi, x; \theta_D\right) &= prob^t_{di_t}, \end{aligned} \tag{19}$$

where $W_o \in \mathbb{R}^{d\times 2}$ is the parameter matrix, di_t is the direction of e_{π_t}.

D. Optimizing Parameters with Reinforcement Learning

As shown in Equation (4), the entire model of NSRL parameterizes a routing policy. Given a CARP instance x, the training objective for the entire model is to minimize the expected cost of the solution route $y = (\pi, di)$ where π and di are the outputs of Set2Seq and DI respectively,

$$L_{\theta_G,\theta_S,\theta_D}(x) = \mathbb{E}_{y\sim p(\cdot|x;\theta_G,\theta_S,\theta_D)} C(y) \tag{20}$$

However, optimizing the two policies simultaneously with $L_{\theta_G,\theta_S,\theta_D}(x)$ is quite difficult. Therefore, we replace the single objective function with two separated objective functions,

$$L_{\theta_G,\theta_S}(x) = \mathbb{E}_{\pi\sim p(\cdot|x;\theta_G,\theta_S)} C((\pi, di^0)) \tag{21}$$

$$L_{\theta_D}(x) = \mathbb{E}_{di\sim p(\cdot|x;\theta_D)} C((\pi, di)) - L_{\theta_G,\theta_S}(x) \tag{22}$$

, where di^0 is a zero vector and (π, di^0) is the initial solution given by Set2Seq.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

TABLE I
INFORMATION ABOUT 8 BENCHMARK SETS. $|V|$ IS THE NUMBER OF NODES, $|E|$ IS THE NUMBER OF EDGES, $|T|$ IS THE NUMBER OF TASK EDGES, Q THE THE CAPACITY OF EACH VEHICLE, $demand$ IS THE SERVING DEMAND ON EACH EDGE.

Name	$\|V\|$	$\|E\|$	$\|T\|$	Q	$demand$
Beijing30train	13-31	20-30	5-26	20	1-10
Beijing50train	20-51	30-50	5-40	20	1-10
Beijing30test	13-31	20-30	6-25	20	1-10
Beijing50test	22-51	30-50	8-39	20	1-10
London40	23-41	20-40	9-32	20	1-10
London60	40-61	30-60	14-44	20	1-10
Paris40	17-41	20-40	8-32	20	1-10
Paris60	36-61	30-60	16-43	20	1-10

The well-known REINFORCE algorithm [12] is used in NSRL for optimizing the parameters θ_G, θ_S, θ_D based on the two objective functions. The calculations of gradients are formulated as,

$$\nabla_{\theta_G,\theta_S} L_{\theta_G,\theta_S}(x) = \mathbb{E}_{\pi\sim p(\cdot|x;\theta_G,\theta_S)}\Big[\Big(C((\pi,di^0)) - b_S(x)\Big)\nabla_{\theta_G,\theta_S} log\ p(\pi|x;\theta_G,\theta_S)\Big], \tag{23}$$

$$\nabla_{\theta_D} L_{\theta_D}(x) = \mathbb{E}_{di\sim p(\cdot|x;\theta_D)}\Big[\Big(C((\pi,di)) - C((\pi,di^0)) - b_D(x)\Big)\nabla_{\theta_D} log\ p(di|x;\theta_D)\Big], \tag{24}$$

where $p(\pi|x;\theta_G,\theta_S)$ can be computed by Equation (2) and Equation (14), $p(di|x;\theta_D)$ can be computed by Equation (3) and Equation (19). $b_S(x)$ and $b_D(x)$ are two baseline functions that estimate expected rewards for Set2Seq and DI to reduce the variance of the gradients during training. The formulations of them are:

$$b_S(x) = \sum\nolimits_{i=0}^{T} S_{e_i} W_{bs}, \tag{25}$$

$$b_D(x) = \sum\nolimits_{i=0}^{T} S_{e_i} W_{bd}, \tag{26}$$

$W_{bs} \in \mathbb{R}^{l\times 1}, W_{bd} \in \mathbb{R}^{l\times 1}$ are parameters. Supervised learning are used to train b_S and b_D and the loss functions for training are:

$$L_{bs}(x) = \Big[b_S(x) - dc\left((\pi,di^0)\right)\Big]^2, \tag{27}$$

$$L_{bd}(x) = \Big[b_D(x) - \big(dc\left((\pi,di^0)\right) - dc\left((\pi,di)\right)\big)\Big]^2. \tag{28}$$

IV. EXPERIMENTS

The numerical experiments are conducted to investigate the behavior of the proposed method by comparisons with two methods. The first one is MAENS [22], a well-known and powerful population-based heuristic search algorithm for CARP. The second method, called VRP-RL [7] for convenience, uses a DNN model trained by reinforcement learning to solve VRP that is slightly different from CARP. To solve CARP, we integrate the graph transformation, the GCN model, and the DI model into VRP-RL.

TABLE II
HYPERPARAMETERS IN MAENS.

Name	Description	Value
psize	Population size	10
ubtrial	Maximum trials for generating initial solutions	5
opsize	No. of offspring generated in each generation	60
p_{ls}	Probability of carrying out local search	0.2
max_enssize	Maximal ENS neighborhood size	50
n_iter	Number of iterations	50

A. Benchmark Sets

We synthesized 8 benchmark sets, namely Beijing30train, Beijing50train, Beijing30test, Beijing50test, London40, London60, Paris40, Paris60, in which the first two each containing 1 million samples are used for training model and the rest each containing 10000 samples are used for evaluating the generalization performance of the model. In each benchmark set, instances are synthesized based on subgraphs extracted from the roadmap[1] of Beijing or London or Paris according to the name of the benchmark set. For each instance, the capacity Q of the vehicle is fixed at 20 and the demand on each task edge is uniformly sampling from integers $\{1, 2, ..., 10\}$. The deadheading cost of each edge is the actual distance of the edge traveled on the roadmap. The detail descriptions of those benchmark sets are shown in Table I

B. Hyperparameters Configuration

For both NSRL and VRP-RL, the three hidden layers of the GCN have 64, 16, and 8 neurons, respectively. In Set2Seq, the weight matrix of the encoder is 8×128 (8 is equivalent to the dimension of edge embeddings produces by GCN, 128 is equivalent to the number of hidden units used in the decoder), the decoder uses a single layer unidirectional GRU with 128 hidden units. Both the encoder and the decoder in DI use a single layer bidirectional GRU with 128 hidden units. The decoder in VRP-RL uses a single layer unidirectional GRU with 128 hidden units. For Both NSRL and VRP-RL, the parameters are initialized with Xavier initialization [23] and are trained by the Adam optimizer [24] with the initial learning rate 5×10^{-4}. The L2 norm of gradients are clipped to 2.0. The size of the mini-batch for training is 128. The dropout rate for GRUs used in NSRL and VRP-RL are set as 0.2. Both NSRL and VRP-RL are implemented with Pytorch [25], a open source deep learning software platform with Python API. For MAENS, the official implementation[2] in C++ is used. The hyperparameters of MAENS are summarized in Table II.

For convenience, we call the solutions generated by MAENS as the reference solutions, denoted as Y^*, and the solutions generated by DNN-based methods as the predicted solutions, denoted as Y'. $Y_i^* \in Y^*$ and $Y_i' \in Y'$ are solutions of the same instance. In order to evaluate the quality of

[1] https://www.openstreetmap.org
[2] http://staff.ustc.edu.cn/ ketang/codes/MAENS.zip

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

TABLE III
AVERAGE COSTS AND OPTIMALITY GAPS OF SOLUTIONS ON SIX BENCHMARK SETS. THE RESULTS OF BEIJING30TEST, LONDON40, AND PARIS40 ARE PRODUCED BY MODELS TRAINED ON BEIJING30TRAIN, AND THE RESULT OF BEIJING50TEST, LONDON50, AND PARIS50 ARE PRODUCED BY MODELS TRAINED ON BEIJING50TRAIN. VALUES FOR OPTIMALITY GAP ARE GIVEN IN FORMS OF "MEAN±STD".

Name	MAENS		VRL-RL		NSRL	
	cost	optimality gap	cost	optimality gap	cost	optimality gap
Beijing30test	3292.1252	\	4175.2275	28.38%±19.03%	4028.5068	**23.65%**±17.76%
London40	1394.6375	\	1809.2670	31.55%±17.86%	1768.3320	**28.37%**±16.90%
Paris40	1428.3579	\	1855.8007	31.82%±18.13%	1808.6368	**28.38%**±17.22%
Beijing50test	6341.5010	\	8233.0176	31.69%±19.69%	8164.5410	**30.15%**±19.04%
London60	2915.7185	\	3874.3364	34.95%±17.93%	3866.2161	**34.34%**±17.35%
Paris60	2610.5215	\	3488.4187	36.00%±18.31%	3472.8291	**35.11%**±17.83%

TABLE IV
COMPARISON OF AVERAGE RUNNING TIME ($\times 10^{-3}$ SECONDS PER INSTANCE). VALUES ARE GIVEN IN FORMS OF "MEAN±STD".

Name	MAENS	VRL-RL		NSRL	
	CPU	CPU	GPU	CPU	GPU
Beijing30test	215±304	200±42	46±18	140±30	32±12
London40	543±736	261±44	73±13	184±33	75±15
Paris40	599±823	261±44	57±21	182±32	65±22
Beijing50test	833±1299	297±65	77±19	211±48	75±24
London60	2088±2390	391±56	105±14	272±41	104±17
Paris60	2310±2579	389±56	99±16	269±41	72±26

solutions given by different methods, an evaluation metric named **optimality gap** is used, i.e.,

$$\textbf{optimality gap} = \frac{100\%}{|Y'|} \sum_{i \in |Y'|} \frac{C(Y_i')}{C(Y_i^*)} - 1. \qquad (29)$$

Note that optimality gap defined here is only meaningful for NSRL and VRP-RL.

C. Results and Analysis

The experiments are divided into two groups according to the problem scale. In the first group, models are trained on Beijing30train and evaluated on Beijing30test, London40, and Paris40. In the second group, models are trained on Beijing50train and evaluated on Beijing50test, London60, and Paris60.

Table III shows the average costs of solutions given by three methods and the optimality gaps of the two DNN-based methods. In terms of the quality of solutions, both NSRL and VRP-RL do not surpass MAENS due to the complexities of CARP. The gaps between solutions generated by NSRL and reference solutions are not worse than 24% on instances with graphs of 20-30 edges and not worse than 31% on instances with graphs of 30-50 edges, which demonstrates the proposed DNN-based model trained by reinforcement learning has certain ability to solve CARP instances. Besides, on all of six benchmark sets, NSRL shows better solution qualities than VRP-RL, which indicates the competitiveness of the proposed method. The graphs of instances in London40, Paris40, London50, and Paris50 are extracted from roadmaps that different from the roadmap used in training sets, which indicates data distributions of these testing sets are not consistent with the training data distributions. However, the solutions of instances in these 4 sets are not significantly worse than Beijing30test and Beijing50test of which the distributions are consistent with the training data distributions, meaning that NSRL has a good generalization performance.

Table IV shows the average running time of two neural solvers and MAENS for solving an instance. Due to using different program languages (Python vs C++), the comparison is just indicative, not rigorous. The solving speed of NSRL is significantly faster than MAENS, especially on GPU. With the increase of the problem scales, the advantage of NSRL on solving speed is more and more significant. Besides, compared with MAENS, the standard deviations of the running time of NSRL are quite small, which means NSRL has more stable running time on solving different instances with similar scales.

V. CONCLUSION

This paper presented a deep neural network based solver optimized by reinforcement learning (NSRL) for solving CARP. To deal with the complexities of CARP, the proposed method introduced a Graph Convolutional Network to parse the information graph of CARP and deploy an encoder-decoder model to indicate the directions of the edges in the solution. NSRL divided the solving for CARP into two polices, one for generating the initial route, and the other for generating directions for edges in the initial route. REINFORCE algorithm with two linear baselines optimized the two policies. Experiment results showed NSRL can generate solutions not worse than the optimal solutions significantly within the running time that much fewer than traditional heuristic algorithm used. NSRL also achieved good generalization performance on unseen instances that quite different from the training data.

For future work, we plan to extend the proposed method to solve CARP instances with larger scales that close to real-world cases. Additionally, exploring the method that generalizes the proposed model to more extensive routing problems is another direction.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.

VI. Acknowledgments

This work was supported by the Natural Science Foundation of China (61672478 and 61806090), Shenzhen Peacock Plan (Grant No. KQTD2016112514355531), the Program for University Key Laboratory of Guangdong Province (Grant No. 2017KSYS008), and the Science and Technology Innovation Committee Foundation of Shenzhen (Grant No. ZDSYS201703031748284). The corresponding author is Guiying Li.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, Las Vegas, NV, United States, 2016, pp. 770–778.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, United states, 2017, pp. 5998–6008.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529, 2015.

[4] O. Vinyals, M. Fortunato, and N. Jaitly, "Pointer networks," in *Advances in Neural Information Processing Systems 28*, Montreal, QC, Canada, 2015, pp. 2692 – 2700.

[5] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," *arXiv preprint arXiv:1611.09940*, 2016.

[6] H. Dai, E. B. Khalil, Y. Zhang, B. Dilkina, and L. Song, "Learning combinatorial optimization algorithms over graphs," in *Advances in Neural Information Processing Systems 30*, Long Beach, CA, United states, 2017, pp. 6349 – 6359.

[7] M. Nazari, A. Oroojlooy, L. Snyder, and M. Takac, "Reinforcement learning for solving the vehicle routing problem," in *Advances in Neural Information Processing Systems 31*, Montreal, QC, Canada, 2018, pp. 9861–9871.

[8] H. A. Eiselt, M. Gendreau, and G. Laporte, "Arc routing problems, part ii: The rural postman problem," *Operations Research*, vol. 43, no. 3, pp. 399–414, 1995.

[9] W.-L. Pearn, A. Assad, and B. L. Golden, "Transforming arc routing into node routing problems," *Computers & Operations Research*, vol. 14, no. 4, pp. 285 – 288, 1987.

[10] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Advances in Neural Information Processing Systems 29*, Barcelona, Spain, 2016, pp. 3844–3852.

[11] O. Vinyals, S. Bengio, and M. Kudlur, "Order matters: Sequence to sequence for sets," in *International Conference on Learning Representations*, Caribe Hilton, San Juan, Puerto Rico, 2016, pp. 1 – 11.

[12] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, no. 3-4, pp. 229–256, 1992.

[13] J. J. Hopfield and D. W. Tank, ""neural" computation of decisions in optimization problems," *Biological Cybernetics*, vol. 52, no. 3, pp. 141–152, 1985.

[14] R. Durbin and D. Willshaw, "An analogue approach to the travelling salesman problem using an elastic net method," *Nature*, vol. 326, no. 6114, p. 689, 1987.

[15] J. C. Fort, "Solving a combinatorial problem via self-organizing process: An application of the kohonen algorithm to the traveling salesman problem," *Biological Cybernetics*, vol. 59, no. 1, pp. 33–40, 1988.

[16] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems 12*, 1999, pp. 1057–1063.

[17] H. Dai, B. Dai, and L. Song, "Discriminative embeddings of latent variable models for structured data," in *33rd International Conference on Machine Learning*, vol. 6, New York City, NY, United States, 2016, pp. 3970 – 3986.

[18] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[19] K. Tang, J. Wang, X. Li, and X. Yao, "A scalable approach to capacitated arc routing problems based on hierarchical decomposition," *IEEE Transactions on Cybernetics*, vol. 47, no. 11, pp. 3928–3940, 2017.

[20] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, Doha, Qatar, 2014, pp. 1724 — 1734.

[21] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Advances in Neural Information Processing Systems 27*, Montreal, QC, Canada, 2014, pp. 3104 – 3112.

[22] K. Tang, Y. Mei, and X. Yao, "Memetic algorithm with extended neighborhood search for capacitated arc routing problems," *IEEE Transactions on Evolutionary Computation*, vol. 13, no. 5, pp. 1151–1166, 2009.

[23] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, vol. 9, Chia Laguna Resort, Sardinia, Italy, 2010, pp. 249–256.

[24] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations*, 2015, pp. 1 – 16.

[25] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.

Authorized licensed use limited to: China Agricultural University. Downloaded on September 09,2026 at 10:32:11 UTC from IEEE Xplore. Restrictions apply.